

Code report

By Team:

Go With the Flow

Members:

Jacob Lebkuecher (904026388), Ryan Giacoboni (904003021), Ethan Van Brunt (904031060)

This program is a discrete-time simulation of TCP's sliding window protocol, demonstrating both congestion control (TCP Tahoe) and flow control. A sender node and receiver node exchange frames and acknowledgments across a simulated lossy channel, where frames and ACKs may be corrupted with a configurable probability. The sender adjusts its congestion window (cwnd) based on incoming ACKs and timeout events, while the receiver advertises its available buffer space (rwnd) on every ACK. After the simulation completes, three matplotlib graphs visualize the congestion window over time, the interaction between flow control and buffer usage, and the cumulative ACKs received by the sender.

How to run it

The program requires Python 3 and the matplotlib library. Install matplotlib using the command:

pip install matplotlib

- 1.) Open a terminal of your choice (Command Prompt, terminal, etc.)
- 2.) Navigate to the directory containing tcp_simulation.py
- 3.) Run the command: python tcp_simulation.py
- 4.) Input your parameters... Following the template below:
"Input Parameters: <duration: int> <sws: int> <rws: int> <error: int> <timeout: int>"
- 5.) Press Enter

Example Test:

C:\directory_to_python_file>python tcp_simulation.py

Input parameters: 16 6 6 0.05 3

Parameters: duration=16 sws=6 rws=6 error=0.05 timeout=3

After the simulation runs, the program prints a tabular log of every timestep to the console and displays a three panel matplotlib graph.

Output

The simulation was run with the following parameters: duration=300, sws=100, rws=100, error=0.03, timeout=30. The full output spans 300 timesteps. The excerpt below shows timestamps 1-25 that are printed in the terminal.

Each row of the output represents a single timestep. The columns are organized as:

timestamp | frame_sent | lar | lfs | lfr | laf | ack | buffer

Code report

1		—	—	0		0	100	0	()
2		1	0	1		0	100	—	()
3		2	0	2		1	101	1	()
4		3	1	3		2	102	2	()
5		4	2	4		3	103	3	()
6		5	3	5		4	104	4	()
7		6	4	6		5	105	5	()
8		7	5	7		6	106	6	()
9		8	6	8		7	107	7	()
10		9	7	9		8	108	8	()
11		10	8	10		9	109	9	()
12		11	9	11		10	110	10	()
13		12	10	12		11	111	11	()
14		13	11	13		12	112	12	()
15		14	?	14		13	113	?	()
16		15	?	15		14	114	?	()
17		16	14	16		15	115	15	()
18		17	15	17		16	116	16	()
19		18	16	18		17	117	17	()
20		19	17	19		18	118	18	()
21		20	18	20		19	119	19	()
22		21	?	21		20	120	?	()
23		22	?	22		21	121	?	()
24		23	21	23		22	122	22	()
25		24	22	24		23	123	23	()

Alongside the table, a graph is created and displayed:

